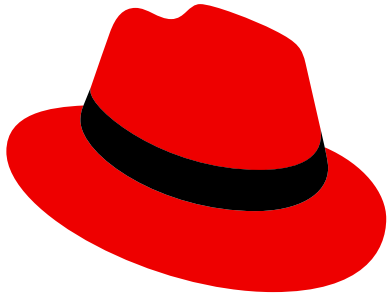
Container Build Pipeline Demo

Four RHEL Deployment Paradigms



Version 1.1 | 2026-06-18

👋 Today's Agenda

1. **Architecture Overview** - What we built
2. **Security-First Build Pipeline** - How we build it
3. **UBI Containers** - RHEL Universal Base Images
4. **RHHI Containers** - Red Hat Hardened Images (RHHI)
5. **UBI bootc** - RHEL Image Mode (Bootable Containers)
6. **RHHI bootc** - Red Hat Hardened Images Bootable OS
7. **Comparison & Trade-offs** - When to use each
8. **Live Demo** - See it in action
9. **Q&A**

What We Built

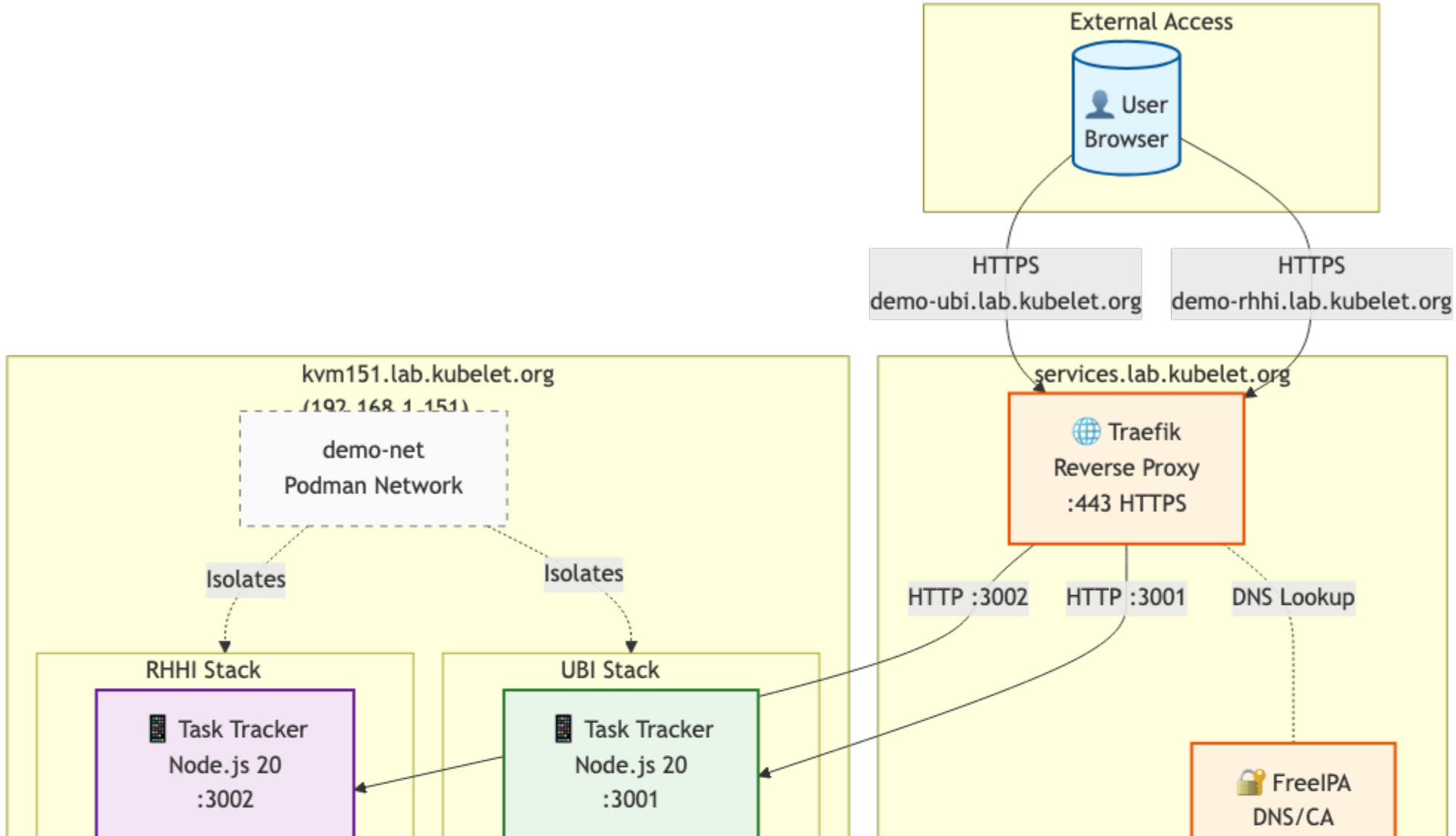
Simple Task Tracker Application

- **Frontend**: Node.js 20 + Express.js
- **Backend**: PostgreSQL database
- **Purpose**: CRUD operations (Create/Read/Update/Delete tasks)
- **Deployment**: Podman + systemd quadlets
- **Routing**: Traefik reverse proxy with TLS
- **DNS**: FreeIPA integration

Built FOUR ways:

- **UBI containers**: RHEL Universal Base Images (enterprise)
- **RHHI containers**: Red Hat Hardened Images (distroless)
- **UBI bootc**: RHEL Image Mode (bootable OS)

System Architecture



Four RHEL Deployment Tracks

Modern deployment paradigms:

1. **Container-native** - UBI & RHHI (Podman + systemd quadlets)
2. **Image mode / bootc** - Bootable OS images (immutable infrastructure)
3. **Traditional** - RPM packages (legacy approach)

This demo covers tracks 1 & 2 with both UBI and RHHI:

- **UBI containers**: Enterprise RHEL-based application containers
- **RHHI containers**: Minimal distroless application containers
- **UBI bootc**: RHEL-based bootable OS with application
- **RHHI bootc**: Fedora-based minimal bootable OS with application
- Same application, four deployment models

Security-First Build Pipeline

Every build goes through:

1. **Multi-stage Build** - Build tools stay in builder, not production
2. **npm audit** - Scan for HIGH/CRITICAL vulnerabilities (FAILS build)
3. **Pre-commit Checks** - Gitleaks blocks commits with secrets
4. **Trivy CVE Scan** - Post-build vulnerability scanning (HIGH/CRITICAL)
5. **SBOM Generation** - CycloneDX format for supply chain transparency
6. **Security Gates** - No vulnerable images reach production

Philosophy: Security is not optional, it's part of the build process.

Two Ways to Build

Local vs CI/CD pipelines

- **Local**: Manual (`make ubi`) - Fast iteration, offline capable
- **CI/CD**: Automatic (git push) - Team visibility, enforced gates
- **Same security gates**: npm audit, Trivy, SBOM
- **Different execution**: Podman locally, Docker Buildx in CI

Best Practice: Local for development, CI/CD for validation

Pipeline Comparison

Feature	Local	CI/CD
Execution	Manual (`make ubi`)	Auto (on push)
Speed	Fast (~10 min)	Slower (~12 min + queue)
Security Gates	Same (npm audit, Trivy, SBOM)	Same
Artifact Storage	Local only	90 days (GitHub)
Security Dashboard	Terminal output	GitHub Security tab
Team Visibility	Local only	All team members
Offline	Yes	Requires internet
Cost	Free (local CPU)	GitHub Actions minutes

Best practice: Local for development, CI/CD for validation

Security Gates Example

```
# Gate 1: npm audit (FAILS on HIGH/CRITICAL)
RUN npm audit --production --audit-level=high || \
    (echo "ERROR: Vulnerabilities found" && exit 1)

# Gate 2: Secret scan (FAILS on secrets found)
trivy image --scanners secret ghcr.io/demo:latest

# Gate 3: CVE scan (reports but doesn't block)
trivy image --severity HIGH,CRITICAL ghcr.io/demo:latest

# Gate 4: SBOM generation
trivy image --format cyclonedx --output sbom.json

# Result: Only secure images without secrets deployed
```

UBI Approach: RHEL Universal Base Images

What is UBI?

- **Base OS**: Red Hat Enterprise Linux 9
- **Support**: 10-year lifecycle, enterprise backing
- **Package Manager**: microdnf available in runtime
- **Use Case**: Production, regulated industries

Our UBI Stack

```
# Builder: ubi9/nodejs-20 (full, ~200MB)
FROM registry.access.redhat.com/ubi9/nodejs-20:latest AS builder

# Runtime: ubi9/nodejs-20-minimal (~150MB)
FROM registry.access.redhat.com/ubi9/nodejs-20-minimal:latest
```

UBI Benefits

Enterprise Support

- 10-year lifecycle (RHEL 9 → 2032)
- Security patches from Red Hat
- FIPS 140-2 compliance available
- Battle-tested in Fortune 500 companies

Operational Flexibility

- Package manager available (`microdnf`)
- Shell access for debugging (`podman exec -it bash`)
- Familiar RHEL tooling
- Easy to add runtime dependencies

UBI Trade-offs

Larger image size:

- Runtime: ~150MB (Node.js minimal)
- More packages = more disk/network usage

More CVEs:

- 7 CVEs in our UBI webapp (RHEL 9.7 base)
- Package manager adds vulnerabilities
- Requires ongoing vulnerability management

Result: Excellent for production, but not minimal

RHHI Approach: Red Hat Hardened Images (RHHI)

What is (RHHI)?

- **Base OS**: Fedora (cutting-edge)
- **Philosophy**: Distroless = minimal attack surface
- **Package Manager**: **NONE** (distroless runtime)
- **Use Case**: Microservices, security-first, edge

Our RHHI Stack

```
# Builder: hi/nodejs:20-builder (Fedora, ~85MB)
FROM registry.access.redhat.com/hi/nodejs:20-builder AS builder

# Runtime: hi/nodejs:20 (distroless, ~44MB)
FROM registry.access.redhat.com/hi/nodejs:20
```

RHHI Benefits

Ultra-Minimal Size

- Runtime: **44MB** (Node.js) vs 150MB UBI = **70% smaller**
- Runtime: **56MB** (PostgreSQL) vs 120MB UBI = **53% smaller**
- Faster image pulls, less disk space
- Faster container startup

Distroless Security

- **NO package manager** (no dnf, no microdnf)
- **NO shell** (no bash, no sh)
- **Minimal CVEs**: 17 for Node.js, **0 for PostgreSQL** 🎉
- Smaller attack surface

RHHI Trade-offs

No runtime flexibility:

- Cannot install packages at runtime
- Cannot exec into shell (distroless)
- Everything must come from builder stage

Less enterprise support:

- Community-driven (not Red Hat supported)
- Shorter lifecycle (Fedora rolling)

Result: Excellent for security, but less operational flexibility

UBI bootc: RHEL Image Mode / Bootable Containers

What is UBI bootc?

- **Type**: Bootable OS image (can run as OS or container)
- **Base OS**: RHEL 9 Image Mode bootc
- **Philosophy**: OS + application as single versioned artifact
- **Deployment**: Bare metal, VM (boots as OS), or container runtime

Our UBI bootc Stack

```
FROM registry.redhat.io/rhel9/rhel-bootc:latest

# Install PostgreSQL + Node.js in OS
RUN dnf install -y postgresql-server nodejs

# Copy application to /opt
COPY src/ /opt/demo-app/
```

UBI bootc Benefits

Immutable infrastructure:

- OS + app versioned together as single image
- Reproducible deployments

Atomic updates:

- `bootc upgrade` updates entire system
- Automatic rollback on failure

Edge ready:

- Single image for remote locations
- No package management needed

UBI bootc Trade-offs

Much larger:

- ~2.25 GB (full RHEL OS with kernel, systemd, network)
- 54 CVEs (full OS surface area)

Requires reboot:

- Updates need system restart

Not multi-tenant:

- One OS instance per system, single app only

Result: Excellent for edge/appliances, but heavyweight for general apps

RHHI bootc: (RHHI) Bootable OS

What is RHHI bootc?

- **Type**: Minimal bootable OS image
- **Base OS**: Red Hat Hardened Images Community (Fedora-based)
- **Image**: `quay.io/hummingbird-community/bootc-os`
- **Philosophy**: Minimal immutable OS + application
- **Deployment**: Bare metal, VM, or container runtime

Our RHHI bootc Stack

```
FROM quay.io/hummingbird-community/bootc-os:latest

# Install Node.js (minimal Fedora packages)
RUN dnf install -y nodejs

# Copy application
```

RHHI bootc Benefits

Minimal bootable OS:

- **1.11 GB** (vs 2.25 GB UBI bootc) = **51% smaller**
- **0 CVEs** 🎉 (minimal surface area)
- Builds in **12 seconds** (vs 8+ minutes for UBI bootc)

Fedora-based:

- Fast updates
- Modern packages
- Multi-arch (x86_64 + aarch64)

Same bootc advantages:

- Atomic updates, immutable infrastructure

RHHI bootc Trade-offs

Smaller than UBI bootc, but still large:

- 1.11 GB (full OS, though minimal)

Fedora-based (not RHEL):

- Community support vs enterprise
- Rolling release cycle

Same bootc limitations:

- Requires reboot for updates
- Not multi-tenant

Result: Minimal bootable OS for edge/appliances with security focus

Four-Way Comparison

Feature	UBI	RHHI	UBI bootc	RHHI bootc
Type	App container	App container	Bootable OS	Bootable OS
Base OS	RHEL 9	Fedora	RHEL 9	Fedora
Total Size	645 MB	**276 MB** ★	2.25 GB	**1.11 GB**
CVEs	7	**0** ★	54	**0** ★
Build Time	~2 min	~1.5 min	~8 min	**12 sec** ★
Pkg Mgr	microdnf	None	None	None
Shell	bash	None	SSH	SSH
Deploy	Podman	Podman	VM/bare metal	VM/bare metal
Updates	Pull/restart	Pull/restart	bootc/reboot	bootc/reboot

When to Choose UBI

Use UBI When:

- **Enterprise support required** (Red Hat backing)
- **10-year lifecycle needed** (RHEL stability)
- **Regulated industries** (FIPS, compliance)
- **Runtime flexibility needed** (install packages on-the-fly)
- **Operational tooling** (need shell access)
- **Legacy compatibility** (RHEL ecosystem)

Examples:

- Banking applications
- Healthcare systems
- Government contracts

When to Choose RHHI

Use RHHI When:

- **Security is paramount** (minimal CVEs)
- **Image size matters** (edge, bandwidth-constrained)
- **Microservices architecture** (many small containers)
- **Modern stack** (no legacy dependencies)
- **Stateless apps** (no runtime config needed)
- **Supply chain transparency** (SBOM-friendly)

Examples:

- Cloud-native microservices
- Edge computing (containers, not OS)
- CI/CD runners

When to Choose UBI bootc

Use UBI bootc When:

- **Immutable infrastructure** (OS + app together)
- **Edge deployments** (remote, single-purpose systems)
- **Enterprise support required** (RHEL backing)
- **Atomic updates needed** (full rollback capability)
- **OS-level control** (kernel, systemd, network stack)
- **Regulatory compliance** (FIPS, certifications)

Examples:

- Enterprise edge locations
- Regulated industry appliances
- Mission-critical single-purpose systems

When to Choose RHHI bootc

Use RHHI bootc When:

- **Minimal footprint critical** (bandwidth/storage constrained)
- **Security-first approach** (0 CVEs, minimal attack surface)
- **Fast updates needed** (12-second builds)
- **Edge deployments** without enterprise requirement
- **Immutable infrastructure** with minimal size

Examples:

- Bandwidth-constrained edge sites
- Security-critical IoT gateways
- High-volume edge deployments (cost savings)
- Digital signage with security focus

Live Demo

NEW: Live Dashboard - http://kvm152.lab.kubelet.org:8889

Watch all four paradigms build in parallel!

```
make demo  # Start dashboard + 4-track parallel builds
```

What You'll See (2x2 Grid):

Live Dashboard (port 8889):

- **2x2 Grid**: Row 1 (UBI + RHHI containers), Row 2 (UBI bootc + RHHI bootc)
- Real-time build progress
- Live log streaming
- Build phases: init → build → scan
- Duration + CVE count tracking

Key Takeaways

1. **Three deployment paradigms** – UBI (enterprise), RHHI (distroless), bootc (immutable OS)
2. **Same security pipeline** – All three go through npm audit, Trivy, SBOM generation
3. **Multi-stage builds eliminate 90% of CVEs** from build tools
4. **Right tool for the job** – Containers for apps, bootc for appliances
5. **Live dashboard** – Parallel builds with real-time progress visualization
6. **Size vs scope** – RHHI smallest (276 MB), bootc largest (~2 GB) but includes OS
7. **Choose based on deployment model**, not just image size

Security Metrics

Stack	Build Time	Size	CVEs (HIGH/ CRITICAL)	Secret Scanning
UBI containers	~2 min	645 MB	7	gitleaks (pre-commit)
RHHI containers	~1.5 min	**276 MB** ★	**0** ★★	gitleaks (pre-commit)
UBI bootc	~8 min	2.25 GB	54	gitleaks (pre-commit)
RHHI bootc	**12 sec** ★★★	**1.11 GB**	**0** ★★	gitleaks (pre-commit)

Key insights:

🔍 Supply Chain Discovery: The npm Problem

Finding: RHHI had MORE CVEs than UBI despite being "minimal"!

Before: 11 CVEs (All from npm)

```
# RHHI runtime included npm + dependencies
/usr/lib/node_modules_20/npm/node_modules/
  ├─ cross-spawn    (CVE-2024-21538)
  ├─ glob           (CVE-2025-64756)
  ├─ minimatch      (3 CVEs)
  └─ tar            (6 CVEs)
Total: 11 HIGH/CRITICAL vulnerabilities
```

Root Cause: (RHHI) `nodejs:20` bundles npm in runtime (no `-minimal` variant exists)

🔍 Supply Chain Discovery: The Fix

Solution: Manually Remove npm

```
# Stage 2: Runtime
FROM registry.access.redhat.com/hi/nodejs:20
USER 0
RUN rm -rf /usr/lib/node_modules_20/npm \
    && rm -f /usr/bin/npm /usr/bin/npx
```

Why this works: npm only needed in builder stage, not production runtime

After: 0 CVEs ✅

```
Report Summary
┌────────┬──────┬─────────────────┐
│ Target │ Type │ Vulnerabilities │
│   -    │  -   │        -        │
```

Impact: 11 → 0 CVEs by removing build tools from production

🔍 Base Image Comparison: UBI vs RHHI

UBI Standard + Node.js Variants

What we're using:

- Base: **UBI9 Standard** (~200 MB - includes dnf/yum)
- Builder: `ubi9/nodejs-20:latest` (Node.js + npm)
- Runtime: `ubi9/nodejs-20-minimal:latest` (Node.js only, **no npm**)
- Total: 245 MB webapp

UBI Variants Available:

- `ubi9` (Standard) - ~200 MB - dnf/yum included
- `ubi9-minimal` - ~90-120 MB - microdnf only
- `ubi9-micro` - ~25-40 MB - **no package manager**

🔍 Base Image Comparison: Why RHHI Had More CVEs

RHHI (RHHI) + Node.js

What exists:

- `hi/nodejs:20-builder` (Node.js + npm + build tools)
- `hi/nodejs:20` (Node.js + npm) ← **Only runtime option**
- ❌ `hi/nodejs:20-minimal` **does not exist**

The Problem:

- No separation between build and runtime Node.js images
- Runtime image includes npm by default
- Bundled npm brings 11 CVEs from supply chain

Our Workaround:

Build Pipeline Highlights

What Makes This Pipeline Production-Ready?

1. **Reproducible Builds** – package-lock.json pinning
2. **Security Scanning** – npm audit + gitleaks + Trivy CVE
3. **SBOM Generation** – Supply chain transparency
4. **Secret Detection** – Pre-commit hooks block commits
5. **Health Checks** – Automated monitoring
6. **Systemd Integration** – Reliable deployment
7. **Secret Management** – Podman native secrets
8. **Automated Testing** – Smoke tests included

🔀 Pipeline Execution: Two Approaches

We demonstrate **both** local and CI/CD pipelines:

1. **Local Pipeline** – Manual, fast iteration
2. **CI/CD Pipeline** – Automated, GitHub Actions

Both run the **same security gates**, different execution models.

Local Pipeline (Manual)

When to use: Development, testing, offline work

```
# Developer runs manually
./scripts/build-demo-ubi.sh    # Build locally
./scripts/scan-demo.sh ubi     # Scan locally
./scripts/deploy-demo-ubi.sh   # Deploy locally
```

Advantages:

- **Fast iteration** - No wait for CI runners
- **Offline capable** - Works without internet
- **Full control** - Debug easily
- **No GitHub runners** - Free

Limitations:

CI/CD Pipeline (GitHub Actions)

When to use: Production, team collaboration, compliance

```yaml
# Automatic on git push
on:
  push:
    paths:
      - 'demo/**'
```

Workflow Steps:

1. **Trigger** - Auto-run on demo/ changes
2. **Build** - Multi-platform Docker Buildx
3. **Secret Scan** - Trivy (exit-code: 1, FAILS build)
4. **CVE Scan** - Trivy (reports to Security tab)
5. **SBOM** - Upload as artifact (90-day retention)

CI/CD Advantages

Automated security:

- Enforced gates (cannot skip scans)
- GitHub Security tab with vulnerability dashboard

Consistency:

- Same environment (Ubuntu runners)
- AMD64 platform builds

Artifacts:

- 90-day SBOM retention
- Auto-push to ghcr.io

CI/CD Limitations

Trade-offs:

- **Slower feedback** - Wait for runners (~5-10 min)
- **Internet required** - Cannot work offline
- **Runner costs** - GitHub Actions minutes (free tier: 2000 min/month)
- **Debugging harder** - Cannot ssh into runners

When to Skip CI/CD:

- Rapid prototyping
- Offline development
- Testing new approaches
- Personal projects

Pipeline Comparison Matrix

Feature	Local Pipeline	CI/CD Pipeline
Execution	Manual (`./scripts/`)	Automatic (git push)
Speed	Fast (minutes)	Moderate (5-10 min)
Security Gates	Optional (can skip)	**Enforced**
Artifacts	Local files only	90-day retention
Team Visibility	None	GitHub Security tab
Offline Work	Yes	No
Cost	Free	Runner minutes
Consistency	Platform-dependent	Ubuntu runners
Debugging	Easy (local access)	Hard (no SSH)

Recommended Workflow

Use BOTH pipelines strategically:

Development Phase:

1. **Local pipeline** - Fast iteration

```
./scripts/build-demo-ubi.sh
./scripts/deploy-demo-ubi.sh
./scripts/demo-tests.sh ubi
```

2. **Iterate quickly** - Fix issues locally

Release Phase:

3. **Git push** - Trigger CI/CD

```
git add demo/
git commit -m "feat: Updated demo"
```

Security Gate Enforcement

Local Pipeline (Honor System):

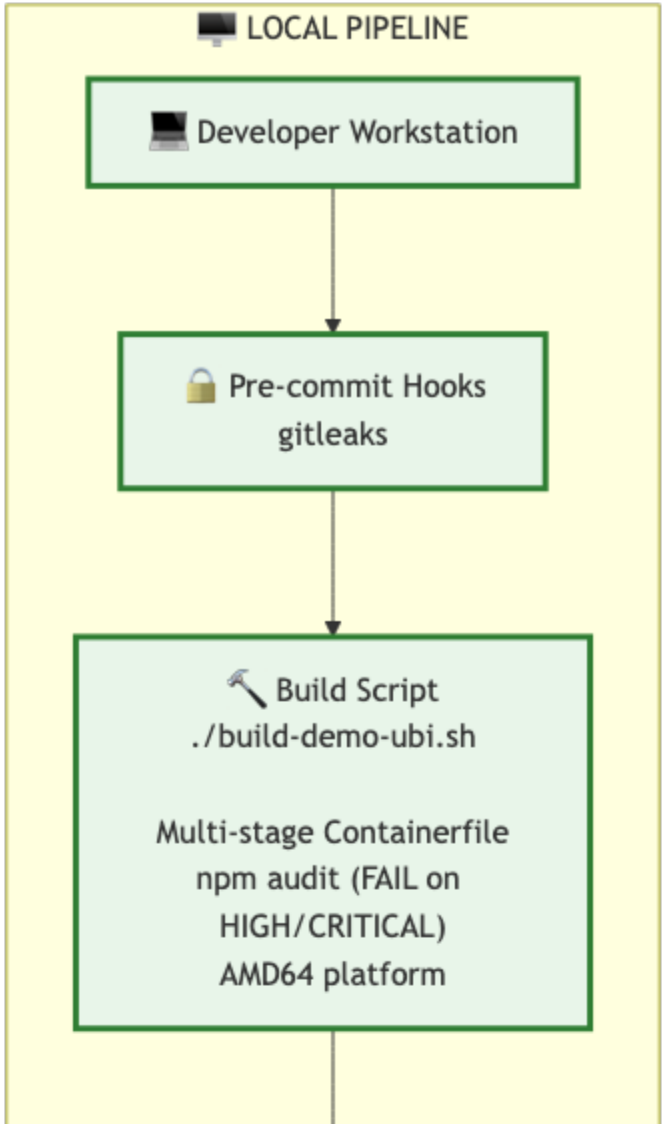
```
# Can skip gates (bad!)
./scripts/build-demo-ubi.sh
# Oops, forgot to scan!
./scripts/deploy-demo-ubi.sh
```

CI/CD Pipeline (Enforced):

```
# Cannot skip - build fails on secrets
- name: Secret Scan
  exit-code: '1'  # FAIL build
  severity: 'HIGH,CRITICAL'
```

Result: CI/CD prevents vulnerable images from reaching production.

CI/CD Workflow Visualization



Resources

Code & Documentation

- **GitHub**: https://github.com/jkirklan/homelab
- **Demo Code**: https://github.com/jkirklan/homelab/tree/main/demo
- **Container Security Guide**: `docs/02-guides/container-security-scanning.md`

(RHHI)

- **Catalog**: https://catalog.hummingbird-project.io
- **API**: https://api-hummingbird.hummingbird-project.io/v1/docs/
- **Source**: https://gitlab.com/redhat/hummingbird/containers

Red Hat UBI

- **Catalog**: https://catalog.redhat.com/software/base-images

Questions?

Topics We Can Dive Into:

- Multi-stage Containerfile patterns
- Trivy scanning setup
- SBOM generation and usage
- Podman quadlet deployment
- Traefik reverse proxy configuration
- FreeIPA DNS integration
- Security incident response (supply chain attacks)
- Distroless debugging techniques

Thank You!

Contact:

- **Demo Code**: https://github.com/jkirklan/homelab/tree/main/demo
- **Deployment Scripts**: https://github.com/jkirklan/homelab/tree/main/demo/scripts
- **Architecture Diagrams**: https://github.com/jkirklan/homelab/tree/main/demo/slides/architecture

Try It Yourself:

```
git clone https://github.com/jkirklan/homelab.git
cd homelab/demo
make help                  # See all targets
make ubi                   # Full UBI pipeline
make rhhi                  # Full RHHI pipeline
make bootc                 # Full bootc pipeline
make demo                  # Dashboard + all in parallel
```